```
#include <Wire.h>
#include <Adafruit_MPU6050.h>
#include <Adafruit_Sensor.h>
#include <ESP32Servo.h>
#include <math.h>

// Components
Adafruit_MPU6050 mpu;
Servo resistanceServo;

//  Pin Assignments
// ESP32 I2C pins for MPU6050 and PWM pin for servo output
const int servoPin = 25;
const int SDA_pin = 21;
const int SCL_pin = 22;

//  Tremor Detection Settings
// Sliding window stores recent gyro magnitudes to smooth response
const int sampleSize = 10;
float tremorData[sampleSize] = {0};
int tremorIndex = 0;

// Timing state for peak-to-peak interval estimation (frequency estimate)
unsigned long lastPeakTime = 0;
unsigned long peakInterval = 0;

// State values used for peak detection and “stillness” tracking
float lastGyroMagnitude = 0;
unsigned long lastTremorTime = 0;

//  Classification thresholds (tuned experimentally)
// Used to only respond to tremors, not intentional motion
const float MIN_TREMOR_THRESHOLD = 0.3;     // Deadband: ignore micro motion/noise
const float MAX_MACRO_MOVEMENT = 6.0;       // Very large magnitude = likely voluntary
const float MIN_FREQUENCY = 5.0;           // Parkinsonian tremor band lower bound
const float MAX_FREQUENCY = 15.0;          // Tremor band upper bound
const float MACRO_MOVEMENT_THRESHOLD = 3.5;  // Low frequency = intentional movement
const unsigned long RESET_TIME = 1000;      // If quiet this long, return to neutral
int currentAngle = 0;

// Smooth Servo Move
// Prevent sudden torque steps that could feel uncomfortable or unsafe.
// This is a “human factors” feature to make actuation feel gradual.
void smoothServoMove(int fromAngle, int toAngle, int stepDelay = 10) {

 // Ignore tiny corrections to reduce jitter
 if (abs(toAngle - fromAngle) < 3) return;
 if (toAngle > fromAngle) {
  for (int pos = fromAngle; pos <= toAngle; pos++) {
   resistanceServo.write(pos);
   delay(stepDelay);
  }
 } else {
  for (int pos = fromAngle; pos >= toAngle; pos--) {
```

```
    resistanceServo.write(pos);
    delay(stepDelay);
   }
  }
  currentAngle = toAngle;
}
void setup() {
  Serial.begin(115200);
  delay(100);
  // Initialize I2C bus on ESP32-defined pins (not default Arduino pins)
  Wire.begin(SDA_pin, SCL_pin);
  // Verify sensor connectivity early (fail-safe behavior)
  if (!mpu.begin(0x68, &Wire)) {
   Serial.println("MPU6050 not detected!");
   while (1);
  }
  Serial.println("MPU6050 Connected.");
  // Sensor configuration:
  // accel range and gyro range set for human motion
  // bandwidth set to 21 Hz to reduce noise while preserving tremor band (~5–15 Hz)
  mpu.setAccelerometerRange(MPU6050_RANGE_8_G);
  mpu.setGyroRange(MPU6050_RANGE_500_DEG);
  mpu.setFilterBandwidth(MPU6050_BAND_21_HZ);
  // Servo setup (50 Hz PWM, calibrated pulse width range)
  resistanceServo.setPeriodHertz(50);
  resistanceServo.attach(servoPin, 500, 2400);
  resistanceServo.write(0);  // start in neutral/low-resistance position
}
void loop() {
  // Read IMU data (accel + gyro + temp) each loop iteration
  sensors_event_t accel, gyro, temp;
  mpu.getEvent(&accel, &gyro, &temp);
  // Feature extraction:
  // Compute magnitude of angular velocity vector to get orientation-independent motion intensity
  float gyroMagnitude = sqrt(pow(gyro.gyro.x, 2) +
                pow(gyro.gyro.y, 2) +
                pow(gyro.gyro.z, 2));
  Serial.print("Gyro magnitude: ");
  Serial.print(gyroMagnitude);
  unsigned long currentTime = millis();
  // Stillness and reset behavior
  // If motion is below threshold for long enough, return servo to neutral.
  // This keeps the device from staying engaged when tremor stops.
  if (gyroMagnitude < MIN_TREMOR_THRESHOLD) {
   if (millis() - lastTremorTime > RESET_TIME) {
    smoothServoMove(currentAngle, 0);
    Serial.println(" | No tremor, servo reset to 0°.");
```

```
  } else {
    Serial.println(" | Low motion, waiting...");
  }
  return;
 }
 // Peak detection
 // Detect significant changes in magnitude to approximate peaks and estimate period.
 if (abs(gyroMagnitude - lastGyroMagnitude) > 0.1) {
  peakInterval = currentTime - lastPeakTime;
  lastPeakTime = currentTime;
  lastTremorTime = currentTime;
 }
 lastGyroMagnitude = gyroMagnitude;
 // Frequency estimation
 // Convert peak-to-peak time interval (ms) to frequency (Hz).
 // Valid intervals correspond roughly to 2–20 Hz; outside that is treated as invalid.
 float frequency = (peakInterval > 50 && peakInterval < 500) ?
           (1000.0 / peakInterval) : 0;
 Serial.print(" | Freq: ");
 Serial.print(frequency);
 Serial.print(" Hz");
 //  Voluntary movement rejection
 // Two rejection rules:
 // 1.) Low-frequency motion is likely intentional reaching/turning, not tremor.
 // 2.) Very large amplitude combined with not-high frequency is treated as macro movement.
 if ((frequency > 0 && frequency < MACRO_MOVEMENT_THRESHOLD) ||
    (gyroMagnitude > MAX_MACRO_MOVEMENT && frequency < 8.0)) {
  Serial.println(" | Voluntary movement ignored.");
  return;
 }
 // Tremor detection path
 // If frequency is in tremor band, record data in a moving window for averaging
 if (frequency >= MIN_FREQUENCY && frequency <= MAX_FREQUENCY) {
  tremorData[tremorIndex] = gyroMagnitude;
  tremorIndex = (tremorIndex + 1) % sampleSize;
  // Moving average smooths instantaneous spikes and produces more stable servo command
  float sum = 0;
  for (int i = 0; i < sampleSize; i++) {
    sum += tremorData[i];
  }
  float avgTremor = sum / sampleSize;
 }
}
```